

Vectorization

How neural networks are implemented efficiently

For loops vs. vectorization

```

x = np.array([200, 17])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])

def dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:,j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out

```

Handwritten notes: **vectorized** (with arrow pointing to the loop), **[1,0,1]** (with arrow pointing to the return value).

```

X = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
B = np.array([-1, 1, 2])

def dense(A_in, W, B):
    Z = np.matmul(A_in, W) + B
    A_out = g(Z)
    return A_out

```

Handwritten notes: **2D array** (under X), **same** (under W), **1x3 2D array** (under B), **all 2D arrays** (under A_in, W, B), **matrix multiplication** (under np.matmul).

Matrix multiplication code

Matrix multiplication in NumPy

$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix}$
 $A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix}$
 $W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix}$
 $Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$

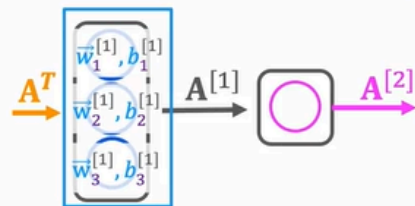
```

A=np.array([[1, -1, 0.1],
            [2, -2, 0.2]])
W=np.array([[3, 5, 7, 9],
            [4, 6, 8, 0]])
AT=np.array([[1, 2],
            [-1, -2],
            [0.1, 0.2]])
AT=A.T
Z = np.matmul(AT, W)
# or
Z = AT @ W

```

Handwritten notes: **transpose** (under AT=A.T), **result** (under Z).

Dense layer vectorized



$$A^T = [200 \quad 17]$$

$$W = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix}$$

$$B = [-1 \quad 1 \quad 2]$$

$$Z = A^T W + B$$

$$\begin{bmatrix} 165 & -531 & 900 \end{bmatrix}$$

$$A = g(Z)$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([[-1, 1, 2]])
```

```
def dense(AT,W,b):
    z = np.matmul(AT,W) + b
    a_out = g(z)
    return a_out
```

```
[[1,0,1]]
```